PRACTICAL EXERCISE 24

Using Generative AI to scaffold frontend code, understand syntax, and implement form validation logic.

PHASE 1: CODE GENERATION

USER PROMPT

"Act as a frontend developer. Create a responsive student registration form using HTML, CSS, and JavaScript. Include name, email, phone number, course, and a submit button. Validate required fields and explain the code for beginners."

Initial Output (JavaScript Validation Snippet):

```
document.getElementById('regForm').addEventListener('submit', function(event) {
event.preventDefault(); // Prevents page reload let name =
document.getElementById('name').value; let phone = document.getElementById('phone').value;
let email = document.getElementById('email').value; // Basic empty field validation
if(name.trim() === '' || phone.trim() === '' || email.trim() === '') { alert('Please fill
out all required fields.'); return; } alert('Registration successful for ' + name); });
```

Beginner Explanation provided by AI:
• **HTML:** Creates the structure (text boxes and buttons).
• **CSS:** Makes it "responsive," meaning the form adjusts its width to look good on both mobile phones and desktop screens.
• **JavaScript:** Listens for the "submit" event. `event.preventDefault()` stops the browser from refreshing immediately, allowing our script to check if the fields are empty (`trim() === ''`) before proceeding.

PHASE 2: DEBUGGING AND ITERATION

Testing Observation (Bug Found):
During testing, we discovered that while the initial code checks if the fields are empty, it does not check the *format*. A user could type letters like "ABCDEF" into the phone number field, and the form would still submit successfully.

USER PROMPT

"I tested the code. The validation only checks if the fields are empty, but the phone number field accepts letters. Please fix the JavaScript validation to only accept exactly a 10-digit number."

Revised Output (Regex Validation Snippet):

```
      document.getElementById('regForm').addEventListener('submit', function(event) {
event.preventDefault(); let phone = document.getElementById('phone').value; // Advanced
validation using Regular Expressions (Regex) let phonePattern = /^\d{10}$/; if
(!phonePattern.test(phone)) { alert('Error: Please enter a valid 10-digit phone number
containing only numbers.'); return; } alert('Validation passed! Registration successful.');
});
```

Key Learning Outcomes

1. Code Generation: AI can instantly scaffold functional boilerplate code (HTML/CSS/JS) to jumpstart projects.
2. Human Testing is Mandatory: AI often provides the simplest possible solution (checking for empty strings) unless prompted with specific edge cases.
3. Iterative Debugging: Instead of writing the complex Regular Expression (`/^\d{10}$/`) manually, the developer can describe the failing test case in plain English, and the AI will patch the logic.